```
$lat = 'ABCDE';
$pol = 'AĄBCĆ';

var_dump(strlen($lat));
var_dump(strlen($pol));

Output:
int(5)
int(7)
```

`$lat` contains the first five letters of the English (Latin) alphabet, and `$pol` contains the first five letters of the Polish alphabet. When we pass those strings to `strlen()`, we get a count of five for `$lat` but seven for `$pol`. It shouldn't come as any surprise (if you've read the rest of the chapter so far) to learn that the Ą and Ć characters have been UTF-8 encoded with multiple bytes. `strlen()` is simply returning the number of bytes in the string; we can confirm this by looking at the byte values for `$lat` and `$pol`:

```
var_dump(bin2hex($lat));
var_dump(bin2hex($pol));

Output:
string(10) "4142434445"
string(14) "41c4844243c486"
```

As you'd expect, `$str` has five bytes of values *0x41–0x44* (the letters A–D in the ASCII range), `$pol` contains a seven-byte sequence *0x41 0xc4 0x84 0x42 0x43 0xc4 0x86*: Ą has been encoded as *0xc4 0x84* and Ć as *0xc4 0x86*. Remember that the `string` data type is just a buffer containing bytes, and non-ASCII characters will be UTF-8 encoded; the behavior of `strlen()` should not be at all surprising. This important differentiation between bytes and information isn't (in my experience) made explicit enough, and even today I still see (and fix) bugs in production systems that result from a lack of multi-byte safety.

The good news is there is a simple solution, which we'll get to in a moment, but first here's another example of multi-byte strings throwing up issues when they're not handled correctly. We'll use my friend Łukasz as the test subject. Sorry, Łukasz, let me buy you a beer to make amends! Just to drive the point home:

```
$me = strtolower('Paul');
$friend = strtolower('Łukasz');
```